Software Metrics Project

Bachelor of Science
Software Engineering Program

953327 Software Metrics

15th October 2026

1. Introduction

This report documents the re-engineering of the SE262 web store, a Bootstrap 5/EJS/Node.js/MySQL e-commerce site, onto a different technology for every component, while still matching the original's functionality. We select and justify each technology using the measurement process taught in the course: define objectives, identify factors, select and weight criteria, exercise the criteria, verify, validate. Abbreviations and ID prefixes used throughout this report are defined in Section 1.1.

1.1. Abbreviations

IDs combine a prefix and a number, for example G-001. Each ID is defined once, in the section shown below, and referenced by ID elsewhere.

Abbreviation	**Meaning**
C-	Selection criterion (Section 4)
Team Member	Teammate
COCOMO	Constructive Cost Model
DET	Data element type
DRF	Django REST Framework
EI	External input
EIF	External interface file
EO	External output
EQ	External inquiry
FP-	Function point inventory item (Section 3.1)
FTR	File type referenced
G-	Project goal (Section 2.1)
ILF	Internal logical file
K-	Constraint (Section 2.3)
KLOC	Thousand lines of code

LEXP	Language and toolset experience, a COCOMO cost driver
ORM	Object-relational mapping
P-	Personal preference (Section 2.2)
RET	Record element type
SE262	The earlier course whose web store is being re-engineered
UFP	Unadjusted function points
v(G)	Cyclomatic complexity
VEXP	Platform experience, a COCOMO cost driver
VK	Virawit Kongthong, team member

2. Objectives

2.1. Goals

ID	**Goal**	**Quality Attribute**	**How can we check it after the rebuild?**
G–001	The rebuilt code is easier to modify than the old EJS/Express code	Maintainability	Section 8.1 – highest v(G), methods above 10, code smells per KLOC
G–002	Fewer defects escape and the code is tested	Reliability	Section 8.1 – bugs per KLOC, statement/branch coverage against quality gate
G–003	The rebuilt store does everything the old one did	Functionality	Section 8.1 and Section 8.5 – functions rebuilt out of the Section 3.1 checklist

2.2. Preferences

ID	Description
P-001	We would find this technology easy to learn without much extra effort.
P-002	We trust this technology because it is mature and widely used in the industry.
P-003	We are familiar with this technology or something close to it.
P-004	We can find resources or documentation easily when we get stuck.

2.3. Constraints

ID	Constraint	Source
K-001	Framework must differ from SE262's project for each component	Project requirements
K-002	The rebuilt store must provide every function of the original project	Project requirements
K-003	SonarQube can analyze the code, and a coverage tool can produce statement and branch coverage that SonarQube can import	Project requirement
K-004	Finished in time for submission and presentation with a team of two people	Project requirement

3. Baseline Measurement of SE262's Project

3.1. Function Points

3.1.1. Data groups (ILF/EIF)

ID	Function/data group	Type	DET	FTR/RET	Complexity	Weight
FP-001	Customer account (profile, address,	ILF	20	4 RET	avg	10

	verification/reset token)					
FP-002	Product catalogue (product, translation, variant, image, category)	ILF	21	5 RET	avg	10
FP-003	Categories	ILF	7	2 RET	low	7
FP-004	Cart	ILF	5	2 RET	low	7
FP-005	Favourites (wishlist)	ILF	3	1 RET	low	7
FP-006	Orders (order, order items)	ILF	24	2 RET	avg	10
FP-007	Product view log	ILF	5	1 RET	low	7
FP-008	Campaign banner (home page hero)	ILF	5	1 RET	low	7
FP-009	Admin accounts	EIF	4	1 RET	low	5

3.1.2. Transactions (EI/EO/EQ)

ID	**Function**	**Type**	**DET**	**FTR**	**Complexity**	**Weight**
FP-010	Sign up (email + details step)	EI	7	1	low	3
FP-011	Verify email from link	EI	2	1	low	3
FP-012	Customer sign in	EI	3	1	low	3
FP-013	Customer sign out	EI	1	0	low	3
FP-014	Forgot password (request reset)	EI	2	1	low	3
FP-015	Reset password	EI	4	1	low	3
FP-016	Add to favourites	EI	2	2	low	3
FP-017	Add to cart	EI	5	2	avg	3

FP-018	Remove cart item	EI	2	2	low	3
FP-019	Change cart item quantity	EI	3	2	low	3
FP-020	Remove cart item	EI	2	2	low	3
FP-021	Place order (save address and purchase)	EI	10	4	high	6
FP-022	Home page (hero banner, new arrivals)	EQ	9	2	avg	4
FP-023	Category landing page (Men/Women/Kids/Baby/Unisex)	EQ	8	2	avg	4
FP-024	All products page	EQ	9	2	avg	4
FP-025	Search products	EQ	14	2	avg	4
FP-026	Product details	EQ	12	3	avg	4
FP-027	View cart	EO	9	2	avg	5
FP-028	Checkout summary	EO	18	3	avg	5
FP-029	View favourites	EQ	9	2	avg	4
FP-030	View profile	EQ	6	1	low	3
FP-031	Category navigation (navbar)	EQ	2	1	low	3
FP-032	Admin sign in	EI	3	1	low	3
FP-033	Admin sign out	EI	1	0	low	3
FP-034	Admin dashboard (sales summary)	EO	16	5	high	7
FP-035	List categories (admin)	EQ	3	1	low	3
FP-036	Create category	EI	2	1	low	3
FP-037	Rename category	EI	3	1	low	3
FP-038	Show/hide category	EI	2	1	low	3

FP-039	Delete category	EI	2	1	low	3
FP-040	List products (admin)	EQ	9	2	avg	4
FP-041	Load product into edit form	EQ	8	2	avg	4
FP-042	Create product (with image upload)	EI	13	2	avg	4
FP-043	Update product	EI	8	2	avg	4
FP-044	List orders (admin)	EQ	8	1	low	3
FP-045	View order details (admin)	EQ	25	2	high	6
FP-046	Edit order shipping address	EI	9	1	low	3
FP-047	Delete order	EI	2	1	low	3
FP-048	Launch campaign (banner, optional email blast)	EI	9	2	avg	4

Type	**Count**	**Points**
ILF	8	65
EIF	1	5
EI	23	76
EO	3	17
EQ	13	50
Total (UFP)	**48**	**213**

3.2. Code Baseline

The SE262 code was analysed with SonarQube (Community build 26.9) on backend/src, frontend/server.js and frontend/public/js, 34 JavaScript files in total. SonarQube does not report the maximum complexity per function, so the v(G) figures come from the lizard tool run over the same files. The baseline has no automated tests, so coverage is 0%. Rates per KLOC use SonarQube's count of non-comment lines.

Measured with SonarQube on the SE262 code. The Rebuild column is filled in Section 8.1.

Metric	**SE262 baseline**	**Rebuild**	**Goal checked**
Statement coverage	0.0% (no tests exist)	98.7%	G-002
Branch coverage	0% (no tests exist)	91.8%	G-002
Bugs per KLOC	0.62 (8 bugs)	0.00 (0 bugs)	G-002
Highest v(G)	66 (updateAdminProduct in product.service.js)	Backend 19 (render_email in common/email_templates.py). Frontend 33 (ProductDetailPage.jsx)	G-001
Methods with v(G) above 10	45 of 713 methods	Backend 13 of 353 functions (3.7%). Frontend 35 of 881 functions (4.0%)	G-001
Code smells per KLOC	9.18 (119 code smells)	0.43 (5 code smells)	G-001
Size (KLOC)	12.97 (12,966 non-comment lines)	11.56 (11,555 non-comment lines)	context for the per-KLOC rates

Rebuild figures are from SonarQube Cloud, dev branch, analysis of 15 October 2026. The scopes differ, so read the comparison with the notes in Section 8.1.

4. Factors and Criteria

4.1. Weights

Each criterion traces to the goals, preferences and constraints in Section 2. Weights range from 1 (minor) to 5 (critical).

ID	Criterion	Traces to	Weight	Why
C-001	Learning and setup effort	P-001, K-004	4	Fixed deadline, two people
C-002	Test and coverage tooling support	G-002, K-003	5	K-003 is a pass/fail process requirement, so weak tooling here blocks the whole project
C-003	Community support and documentation	P-002, P-004	3	Helps solve problems, but is not decisive on its own
C-004	Structure and maintainability support	G-001	4	Main lever for the maintainability goal
C-005	Functional fit with the original store	G-003, K-002	3	Some options make replicating a function harder than others, or some add unnecessary complexity

4.2. Measurement Definitions

Each criterion is defined as a measure with an attribute, a scale and a unit. The validity column states whether the measure really captures the attribute and how we checked.

ID	Entity and attribute type	Attribute, Scale, Unit	Scale Type	Data source	Validity concern and check

C-001	Resource (team), external	Learning and setup effort, LEXP * VEXP effort multiplier, relative factor	Ordinal ratings mapped to ratio multipliers	Section 4.4 ratings	Ratings are self-assessed. Each rating has a short basis, and a small hello-world spike per option checks them.
C-002	Product (framework), external	Coverage tooling support, requirements met out of 4 (a test framework, statement coverage, branch coverage, a report format SonarQube can import), count	Absolute (count)	Official documentation and a spike run	A feature in the documentation may not work in practice. A spike produces a real coverage report, and "met" means it worked.
C-003	Product (framework), external	Community support, Stack Overflow questions with the framework's tag, count	Ratio	Stack Overflow tag counts	A large count can mean more problems rather than more help, and old answers go stale. It is cross-checked against documentation quality, and any disagreement is noted.
C-004	Product (framework), external	Structure support, structural mechanisms met out of 4 (listed for each component in Section 5), count	Absolute (count)	Official documentation	Having a mechanism does not guarantee maintainable code. The real check is the post-rebuild metrics in Section 8.1.
C-005	Product	Functional fit,	Ratio	Sectio	The result

	(framework), external	share of the function groups for that component supported without custom workarounds (groups listed in Section 5 and drawn from Section 3.1), percent		n 3.1 list and official documentation	depends on how functions are grouped, so the groups are fixed before any option is scored.

4.3. Score Rules

Each raw measure is converted to a 1 to 5 score using the rules below, written before any option is scored. The scores are ordinal, but the weighted sum treats them as equally spaced; we accept this simplification. The weighted score of an option is the sum of weight times score over C-001 to C-005, with a maximum of 95.

ID	**Raw scale**	**Score 1**	**Score 2**	**Score 3**	**Score 4**	**Score 5**
C-001	Effort multiplier	1.30 or more	1.15 to 1.29	1.05 to 1.14	0.95 to 1.04	under 0.95
C-002	Requirements met (0 to 4)	0	1	2	3	4
C-003	Rank by question count among the three options	Fewest	not used	Middle	not used	Most
C-004	Mechanisms met (0 to 4)	0	1	2	3	4
C-005	Share of groups supported	under 50%	50 to 69%	70 to 84%	85 to 99%	100%

For C-003 only scores 1, 3 and 5 are used, because three options give only three ranks.

Suitability and preference ratings do not enter the weighted score. They are used in the Close-score check and in the Justification.

4.4. COCOMO Ratings Guide

Team experience is rated with the course's COCOMO cost drivers. Only language and toolset experience (LEXP) and platform experience (VEXP) are used, because the tools (Git and SonarQube) are the same for every option. The effort multiplier of an option is the product of its LEXP and VEXP multipliers, and it is the raw measure for C-001.

Driver	**V.low**	**Low**	**Normal**	**High**
LEXP	1.14	1.07	1.00	0.95
VEXP	1.21	1.10	1.00	0.90

Rating	**LEXP**	**VEXP**
V.low	Never used	Never run or deployed on it
Low	Followed a tutorial or one lab	Ran it locally once
Normal	Finished a project with it	Deployed a project on it
High	Used in more than one project	Deployed and maintained more than one project

Where the two of us differ, the lower rating is used, since both of us must build it.

5. Component Survey

5.1. UI Framework (Original Bootstrap)

5.1.1. Options

Technology

- Option A: Tailwind CSS
- Option B: Fomatic UI
- Option C: Foundation

Overview

- Option A: Utility-first CSS framework that provides small, reusable classes for building custom responsive designs.

- Option B: Component-based UI framework that provides ready-made responsive components with a semantic and human-readable class structure.
- Option C: Responsive CSS framework that provides a grid system and pre-built components for building flexible web interfaces.

5.1.2. Strengths and weaknesses

Option A: Tailwind CSS

- **Strengths:** Highly customizable and flexible, allowing developers to create unique designs without being restricted to predefined components. It provides many utility classes which makes it suitable for both simple and complex web applications.
- **Weaknesses:** It can make HTML code longer because many utility classes may need to be added to each element. Lacks ready-made components, so these must be built manually or added through other libraries.

Option B: Fomantic UI

- **Strengths:** Large collection of ready-made UI components.. Its semantic class naming makes the HTML structure relatively easy to understand and build interfaces quickly.
- **Weaknesses:** Provides less flexibility for highly customized designs compared with utility-first frameworks..

Option C: Foundation

- **Strengths:** Provides a responsive grid system and a range of pre-built UI components for developing responsive web interfaces. Its layout system supports different screen sizes and allows developers to create flexible page structures.
- **Weaknesses:** Relatively larger feature set.

5.1.3. Suitability for project goals

ID	Tailwind CSS	Fomantic UI	Foundation
G–001	**Strong**. Utility classes provide consistent styling and allow individual components to be modified without changing large CSS files. However, excessive utility classes can make markup longer.	**Strong**. Semantic class names and reusable UI components make the interface structure easier to understand and modify.	**Strong**. Its grid system and reusable components provide a consistent structure, making layouts easier to modify and maintain.
G-002	**Partial**. Consistent utility classes can reduce styling inconsistencies and UI implementation errors, but Tailwind does not provide built-in application testing or directly improve test coverage.	**Partial**. Pre-built and reusable components can reduce UI implementation errors, but reliability still depends on separate testing and does not directly provide application-level test coverage.	**Partial**. Pre-built responsive components can reduce common UI implementation errors, but testing and application reliability must be handled separately.
G-003	**Strong**. Provides utilities for responsive layouts, forms, navigation, buttons, product cards, and other interfaces needed to reproduce the existing store's functionality.	**Strong**. Provides ready-made components such as menus, forms, buttons, cards, tables, and navigation that support rebuilding the existing store interface.	**Partial**. Pre-built responsive components can reduce common UI implementation errors, but testing and application reliability must be handled separately.

5.1.4. Suitability for personal preferences

Each of us rates P-001 to P-004 separately from 1 (low) to 5 (high). The Team row is the average of the two ratings. A difference of 2 or more points between us on a preference is noted below the table.

Option	Rater	P-001	P-002	P-003	P-004	Mean
Tailwind CSS	VK	4	5	4	4	4.25
	Team Member	3	5	4	4	4
	Team	3.5	5	4	4	4.13
Fomantic UI	VK	3	2	1	3	2.25
	Team Member	4	2	2	3	2.75
	Team	3.5	2	1.5	3	2.50
Foundation	VK	3	1	2	3	2.25
	Team Member	3	3	2	3	2.75
	Team	3	2	2	3	2.50

Differences of 2 or more points: Foundation P-002 (VK = 1, Team Member = 3).

5.1.5. Team experience

Team experience is rated with the COCOMO levels in Section 4.4, using the lower rating where we differ.

Framework	Rater	LEXP	VEXP	Effort multiplier	Basis
Tailwind CSS	VK	High (0.95)	Normal (1.00)	0.95	Used in two projects. Deployed once, not

					yet deployed and maintained more than once.
	Team Member	Normal (1.00)	Normal (1.00)	1.00	Used and deployed it in a course project.
	Used	Normal (1.00)	Normal (1.00)	1.00	Lower of the VK and Team Member ratings for each driver.
Fomantic UI	VK	V.low (1.14)	V.low (1.21)	1.38	Never used or deployed; no experience at all.
	Team Member	V.low (1.14)	V.low(1.21)	1.38	Never used the framework before.
	Used	V.low (1.14)	V.low(1.21)	1.38	Lower of the VK and Team Member ratings for each driver.
Foundation	VK	V.low (1.14)	V.low (1.21)	1.38	Never used or deployed; no experience at all.

	Team Member	Low (1.07)	Low (1.10)	1.18	Explore a project with this framework.
	Used	V.low (1.14)	V.low (1.21)	1.38	Lower of the VK and Team Member ratings for each driver.

Effort multiplier is the LEXP multiplier times the VEXP multiplier. The Used row is the one that feeds C-001.

5.1.6. Weighted comparison

Each cell shows the raw measure and, in brackets, the score from Section 4.3. The weighted score is the sum of weight times score, with a maximum of 95.

Criterion	Weight	Tailwind CSS	Fomantic UI	Foundation
C-001	4	1.00 (4)	1.38 (1)	1.38 (1)
C-002	5	4/4 (5)	4/4 (5)	4/4 (5)
C-003	3	10,588 questions (5)	45 questions (1)	5,106 questions (3)
C-004	4	4/4 (5)	3/4 (4)	3/4 (4)
C-005	3	5/5, 100% (5)	5/5, 100% (5)	5/5, 100% (5)
Weighted Score	**19**	**91**	**63**	**69**

C-004 mechanisms for the UI component: (1) consistent styling vocabulary/theme, (2) responsive breakpoint/layout tools, (3) state/pseudo-class styling, (4) custom styling without overriding a large predefined component design. These are assessment definitions

proposed for this component. Tailwind meets 4/4; Fomantic UI and Foundation meet 3/4 on this rubric.

C-005 UI groups, fixed against Section 3.1: (1) navigation and home page, (2) product catalogue, categories and detail views, (3) authentication, profile and account forms, (4) favourites, cart and checkout displays, (5) admin tables, forms and dashboard layout. All three systems can style all five; this is presentation fit, not proof that styling frameworks implement behaviour.

C-003 counts: Stack Overflow tag pages surfaced during drafting: tailwind-css 10,588; fomantic-ui 45; zurb-foundation 5,106. Tag choice influences the result: fomantic-ui omits questions under broader semantic-ui tags.

5.1.7. Close-score check

Tailwind CSS scored 91, Foundation 69, and Fomantic UI 63 on the provisional comparison. The leading margin is 22 points, so the result is not close under the proposed rubric. Tailwind leads the others on C-001 (the team's existing experience), C-003 (more questions under the compared Stack Overflow tags), and C-004 (flexible theming and utility-based styling). All three tie on C-002 and C-005. The team preference means are Tailwind 4.13, Fomantic UI 2.50 and Foundation 2.50. The difference of two points on Foundation P-002 is noted above. Preference ratings do not enter the weighted total.

5.1.8. Decision

Selected framework: Tailwind CSS

5.1.9. Justification

Tailwind CSS is selected because it achieved the highest weighted score of 91/95, compared to Foundation (69/95) and Fomantic UI (63/95). It scored maximum points (5/5) across C-002 (4/4 test/coverage requirements met, weight 5), C-003 (10,588 Stack Overflow questions, weight 3), C-004 (4/4 structural mechanisms met, weight 4), and C-005 (100% functional fit across 5/5 UI groups, weight 3). Additionally, under C-001 (weight 4), Tailwind achieved an effort multiplier of 1.00 (score 4) based on normal team experience (LEXP 1.00, VEXP 1.00), whereas Foundation and Fomantic UI both

required very low experience multipliers of 1.38 (score 1). Tailwind also aligned with team preferences, receiving a mean preference score of 4.13 compared to 2.50 for both alternatives.

5.1.10. Verify

ID	What we checked	Result	Evidence
G-001	Is the UI code easier to modify than the old Bootstrap styles?	Met for UI maintain ability	Tailwind utility classes allow inline component styling changes without modifying global CSS files.
G-002	Are UI component layout errors reduced?	Met	SonarQube reported 5 maintainability issues in the rebuilt frontend, compared with 119 code smells.
G-003	Does the UI reproduce all screens from the original store?	Met	All original store screens were rebuilt using Tailwind CSS, including the home page, product catalogue, product details, cart, checkout, user profile, and admin interfaces.
K-001	Does the framework differ from Bootstrap 5?	Met	Tailwind CSS replaces Bootstrap 5.
K-002	Does the UI support all original features?	Met,	All UI features from the original store were reproduced using Tailwind CSS, including navigation, product displays, search and filter interfaces, cart, checkout, user profiles, and admin pages. UI completeness is measured by comparing the implemented interface features against the original requirements checklist in Section 3.1.
K-003	Can SonarQube analyze CSS/UI code?	Met	SonarQube analyzes the frontend source containing

			CSS. The project reports maintainability findings, including CSS color-contrast issues noted in the original report.
K-004	Was the UI rebuild finished on time?	Yes we finished on time	The dev branch has 286 commits from 24 September to 14 October 2026, built by two people

5.1.11. Validate

Tailwind CSS 4.3.3 was validated in the production frontend build. The frontend coverage run passed 345 tests across 32 test files, with 96.93% statement coverage, 91.89% branch coverage, 94.29% function coverage, and 98.99% line coverage. These results show that Tailwind is integrated into a buildable interface and the styling framework works within the project's tested frontend setup.

5.2. Frontend Framework (Original EJS)

5.2.1. Options

Technology

- Option A: Vue
- Option B: React
- Option C: Angular

Overview

- Option A: Progressive frontend framework that provides a component-based architecture for building interactive web interfaces. It offers built-in features such as reactive state management, directives, and routing support through its ecosystem.
- Option B: Component-based JavaScript library for building interactive user interfaces. It uses a declarative approach and can be extended with libraries for routing, state management, and other application features.
- Option C: Full-featured TypeScript-based frontend framework that provides built-in solutions for components, routing, forms, dependency injection, and application structure.

5.2.2. Strengths and weaknesses

Option A: Vue

- **Strengths:** Easy to learn and provides a clear component-based structure. Its reactive system and single-file components make it suitable for developing interactive web applications while keeping HTML, CSS, and JavaScript organized.
- **Weaknesses:** Some advanced features, such as state management and routing, are handled through separate libraries in the Vue ecosystem. It has a smaller ecosystem and market presence compared with React.

Option B: React

- **Strengths:** Flexible component-based architecture with a large ecosystem and strong community support. It can be used with many libraries for routing, state management, and other application requirements.
- **Weaknesses:** React itself focuses mainly on the UI layer, so additional libraries are often required for features such as routing and state management. The large number of available libraries can also make technology choices more complex.

Option C: Angular

- **Strengths:** Flexible component-based architecture with a large ecosystem and strong community support. It can be used with many libraries for routing, state management, and other application requirements.
- **Weaknesses:** React itself focuses mainly on the UI layer, so additional libraries are often required for features such as routing and state management. The large number of available libraries can also make technology choices more complex.

5.2.3. Suitability for project goals

Goal	Vue	React	Angular
G-001	**Strong**. Vue's component-based structure and single-file components make UI code organized and easy to modify.	**Strong**. React's reusable component architecture supports modular code and makes individual UI components easier to modify.	**Strong**. Angular provides a structured component-based architecture that supports organized and maintainable code.

G-002	**Strong**. Vue supports established testing tools and integrates with linting and type-checking tools to help identify defects during development.	**Strong**. React has a large testing ecosystem and supports component testing, linting, and type checking.	**Strong**. Angular provides built-in testing support and a structured development environment that helps detect defects.
G-003	**Strong**. Vue supports reusable components, routing, state management, and API integration needed to rebuild the original store.	**Strong**. React's component ecosystem and supporting libraries can implement the required store functionality.	**Strong**. Angular provides routing, forms, HTTP communication, and other features needed to rebuild the original store.

5.2.4. Suitability for personal preferences

Each of us rates P-001 to P-004 separately from 1 (low) to 5 (high). The Team row is the average of the two ratings. A difference of 2 or more points between us on a preference is noted below the table.

Option	**Rater**	**P-001**	**P-002**	**P-003**	**P-004**	**Mean**
Vue	VK	4	3	5	4	4
	Team Member	4	4	5	4	4.25
	Team	4	3.5	5	4	4.16
React	VK	4	5	4	3	4
	Team Member	3	5	4	4	4
	Team	3.5	5	4	3.5	4
Angular	VK	3	4	1	4	3
	Team Member	3	4	2	4	3.25
	Team	3	4	1.5	4	3.16

Differences of 2 or more points:

5.2.5. Team experience

Team experience is rated with the COCOMO levels in Section 4.4, using the lower rating where we differ.

Framework	**Rater**	**LEXP**	**VEXP**	**Effort multiplier**	**Basis**
Vue	VK	High (0.95)	Normal (1.00)	0.95	Used and deployed in two course projects and a hackathon.
	Team Member	Normal (1.00)	Normal (1.00)	1.00	Used and deployed it in a course project.
	Used	Normal (1.00)	Normal (1.00)	1.00	Lower of the VK and Team Member ratings for each driver.
React	VK	Normal (1.00)	Normal (1.00)	1.00	Used and deployed in a freelance project.
	Team Member	Normal (1.00)	Normal (1.00)	1.00	Completed a React course and developed it into a project.
	Used	Normal (1.00)	Normal (1.00)	1.00	Lower of

					the VK and Team Member ratings for each driver.
Angular	VK	V.low (1.14)	V.low (1.21)	1.38	Never used or deployed.
	Team Member	V.low (1.4)	V.low (1.21)	1.38	Never used the framework before.
	Used	V.low (1.4)	V.low (1.21)	1.38	Lower of the VK and Team Member ratings for each driver.

Effort multiplier is the LEXP multiplier times the VEXP multiplier. The Used row is the one that feeds C-001.

5.2.6. Weighted comparison

Each cell shows the raw measure and, in brackets, the score from Section 4.3. The weighted score is the sum of weight times score, with a maximum of 95.

Criterion	Weight	Vue	React	Angular
C-001	4	1.00 (4)	1.00 (4)	1.38 (1)
C-002	5	4/4 (5)	4/4 (5)	4/4 (5)
C-003	3	107,900 questions (1)	475,173 questions (5)	306,370 questions (3)

C-004	4	4/4 (5)	4/4 (5)	4/4 (5)
C-005	3	5/5, 100% (5)	5/5, 100% (5)	5/5, 100% (5)
Weighted Score	**19**	**79**	**91**	**75**

C-004 mechanisms for the frontend framework: (1) reusable components, (2) state/reactivity mechanism, (3) conditional/list rendering, and (4) component composition/shared-logic patterns. All three frameworks meet 4/4 on this rubric.

C-005 frontend function groups, mapped to Section 3.1: (1) account access and profile, (2) catalogue/search/product views, (3) cart/favourites/checkout, (4) administrative product/category/order CRUD, (5) admin dashboard/campaign UI. All three frameworks support 5/5 using documented ecosystem tools.

C-003 counts: Stack Overflow tag pages surfaced during drafting: vue.js 107,900; reactjs 475,173; angular 306,370.

5.3. Close-score check

React scored 91, Vue 79, and Angular 75 on the provisional comparison. The leading margin between React and Vue is 12 points, so the result is not close under the proposed rubric. React leads on C-003 due to higher community activity on Stack Overflow, while all three frameworks tie on C-001, C-002, C-004, and C-005.

5.4. Decision

Selected framework: React with Vite

5.5. Justification

React 19 with Vite is selected because it achieved the highest weighted score of 91/95, ahead of Angular (75/95) and Vue (79/95). React led decisively on C-003 (community support, weight 3) with 475,173 Stack Overflow questions (score 5), compared to Angular's 306,370 (score 3) and Vue's 107,900 (score 1). On C-001 (learning effort, weight 4), both React and Vue achieved effort multipliers of 1.00 (score 4) due to normal team experience (LEXP 1.00, VEXP 1.00), while

Angular required a higher effort multiplier of 1.38 (score 1). All three frameworks tied with maximum scores (5/5) on C-002 (4/4 coverage requirements met, weight 5), C-004 (4/4 structural mechanisms met, weight 4), and C-005 (100% functional fit across 5/5 groups, weight 3). React also matched the highest team preference mean (4.00 for React and Vue vs. 3.00 for Angular).

5.6. Verify

ID	What we checked	Result	Evidence
G-001	Is the frontend code easier to modify than EJS templates?	Improved on the figures measured	Highest v(G) is 33 against 66 in SE262. 35 of 881 functions are above 10 (4.0%) against 45 of 713 (6.3%). One hotspot remains: ProductDetailPage.jsx at 33. The EJS templates held 617 <% logic tags across 50 files, against 40 production component.
G-002	Is frontend code tested and verified?	Met	345/345 tests passed across 32 files. Coverage: 96.93% statements, 91.89% branches, 94.29% functions, and 98.99% lines. Type checking and production build passed.
G-003	Does the SPA reproduce all views from the original store?	Route coverage met	23 of 23 legacy storefront EJS templates have a React route (32 paths in total). 38 of 38 distinct transactions (FP-010 to FP-048, with FP-020 a duplicate of FP-018) are traced to a route or shared component and a test that asserts the behavior. Tests use mocked API responses,

			so this shows frontend behavior, not live integration.
K-001	Does the framework differ from EJS?	Met	The legacy uses EJS; the rebuilt frontend uses React 19.3.0 and Vite 8.3.3.
K-002	Does the frontend support all original functions?	Met for the 38 distinct transactions	Same trace: 38 traced, 0 route only, 0 gaps. The nine data groups (FP-001 to FP-009) are backend and database items and are checked in those sections.
K-003	Can SonarQube analyze frontend coverage?	Met	Vitest generates coverage data, and the project's SonarQube workflow imports the frontend LCOV report.
K-004	Was frontend rebuild finished on time?	Yes we finished on time	The dev branch has 286 commits from 24 September to 14 October 2026, built by two people

5.7. Validate

We validated the frontend by building and testing the React 19.3.1 and Vite 8.3.3 single-page app styled with Tailwind CSS 4.3.3. The production build and type check passed. The test run passed 345 of 345 tests across 32 test files, with 96.93% statement coverage, 91.89% branch coverage, 94.29% function coverage and 98.99% line coverage. Vitest writes an LCOV report that the CI workflow imports into SonarQube Cloud, and the quality gate passed on dev (14 October 2026). The frontend has 40 production JSX files and about 4,990 lines of JavaScript and 2,020 lines of CSS. ESLint's complexity rule measured 881 functions with a highest cyclomatic complexity of 33, and 35 functions (4.0%) are above 10. The router declares 32 paths (15 storefront, 8 account, 9 admin), and all 23 legacy storefront EJS templates have a corresponding React route.

5.8. Backend Framework (Original Node.js + Express)

5.8.1. Options

Technology

- Option A: FastAPI (Python)
- Option B: Django WSGI (Python)
- Option C: Spring Boot (Java)

Overview

- Option A: Async-first, minimal Python framework built on Starlette and Pydantic; routes and request/response models are declared with type hints.
- Option B: Batteries-included Python web framework (ORM, admin panel, auth) with a REST layer added on top via DRF.
- Option C: Convention-heavy Java framework built on the Spring ecosystem; typically paired with Spring Data JPA for persistence.

5.8.2. Strengths and weaknesses

Option A: FastAPI (Python)

- **Strengths**: Minimal and quick to learn, since routes and request models are declared with Python type hints. It validates requests and generates interactive API documentation automatically, and it supports async handling out of the box.
- **Weaknesses:** It has no built-in ORM, admin panel or authentication, so database access, migrations and login must be assembled from separate libraries. Project structure is left to the developer, which can hurt maintainability without discipline.

Option B: Django WSGI (Python)

- **Strengths:** Batteries included, with an ORM, migrations, authentication, admin panel and built-in protection against common attacks such as CSRF and SQL injection. It is mature and well documented, and the admin panel could cover much of the original store's admin pages.
- **Weaknesses:** WSGI handles one request per worker at a time, so it copes less well with many concurrent requests. It is

opinionated and heavier than needed for a JSON backend, and the REST layer requires adding Django REST Framework.

Option C: Spring Boot (Java)

- **Strengths:** Very mature, with a large ecosystem and strong typing. Dependency injection and a layered structure make code easy to organise and test. Java tooling such as JUnit, JaCoCo and SonarQube integrate well, which suits the coverage and quality gate requirement.
- **Weaknesses:** It has the steepest learning curve and the most configuration and boilerplate of the three, which is a risk for a two-person team on a fixed deadline. Startup time and memory use are higher than for the Python options.

5.8.3. Suitability for project goals

Strong, Partial, Weak, or None will be used to indicate suitability for the project goals. Goal IDs (G-XXX) refer to the tables in sections 2.1.

Goal	FastAPI	Django WSGI	Spring Boot
G-001	**Partial**. Type hints and Pydantic models make the code explicit, but folder structure and layering are left to us to decide.	**Strong**. Enforced structure (model, views, apps), plus an ORM and migrations, keeps changes in predictable places.	**Strong**. Layered design (controller, service, repository) with dependency injection and static typing makes changes easy to isolate.
G-002	**Strong**. Pytest with a built-in test client, and coverage.py can report statement and branch coverage in a format SonarQube imports.	**Strong**. Built-in test runner and test client, and coverage.py with branch mode feeds SonarQube.	**Strong**. JUnit and MockMvc for tests, and JaCoCo gives statement and branch coverage that SonarQube reads natively.
G-003	**Partial**. Everything is possible, but sessions or tokens, password reset, email verification	**Strong**. Auth, password reset, ORM, forms and an admin panel are built in, and DRF covers the JSON	**Strong**. Spring Security, Spring Data JPA and mail support cover every function, though with more

	and the ORM must be assembled from separate libraries.	API.	setup.

5.8.4. Suitability for personal preferences

Each of us rates P-001 to P-004 separately from 1 (low) to 5 (high). The Team row is the average of the two ratings. A difference of 2 or more points between us on a preference is noted below the table.

Option	**Rater**	**P-001**	**P-002**	**P-003**	**P-004**	**Mean**
FastAPI	VK	4	5	4	5	4.5
	Team Member	4	5	3	5	4.25
	Team	4	5	3.5	5	4.38
Django WSGI	VK	3	4	2	4	3.25
	Team Member	3	4	3	4	3.5
	Team	3	4	2.5	4	3.38
Spring Boot	VK	2	5	4	4	3.75
	Team Member	3	5	4	5	4.25
	Team	2.5	5	4	4.5	4

Differences of 2 or more points: none.

5.8.5. Team experience

Team experience is rated with the COCOMO levels in Section 4.4, using the lower rating where we differ.

Framework	**Rater**	**LEXP**	**VEXP**	**Effort multiplier**	**Basis**
FastAPI	VK	Normal (1.00)	Low (1.10)	1.10	Used during a short

					backend internship.
	Team Member	V.low (1.14)	V.low (1.21)	1.38	Never used the framework before.
	Used	V.low (1.14)	V.low (1.21)	1.38	Lower of the VK and Team Member ratings for each driver.
Django WSGI	VK	Low (1.07)	Low (1.10)	1.18	Only played around and used locally.
	Team Member	V.low (1.14)	V.low (1.21)	1.38	Never used the framework before.
	Used	V.low (1.14)	V.low (1.21)	1.38	Lower of the VK and Team Member ratings for each driver.
Spring Boot	VK	Normal (1.00)	Normal (1.00)	1.00	Used and deployed in a course project.
	Team Member	Normal (1.00)	Normal (1.00)	1.00	Used and deployed it in a course project.
	Used	Normal (1.00)	Normal (1.00)	1.00	Lower of the VK and

					Team Member ratings for each driver.

Effort multiplier is the LEXP multiplier times the VEXP multiplier. The Used row is the one that feeds C-001.

5.8.6. Weighted comparison

Each cell shows the raw measure and, in brackets, the score from Section 4.3. The weighted score is the sum of weight times score, with a maximum of 95.

Criterion	Weight	FastAPI	Django WSGI	Spring Boot
C-001	4	1.38 (1)	1.38 (1)	1.00 (4)
C-002	5	4 of 4 (5)	4 of 4 (5)	4 of 4 (5)
C-003	3	7,292 questions (1)	311,126 questions (5)	150,213 questions (3)
C-004	4	2 of 4 (3)	3 of 4 (4)	3 of 4 (4)
C-005	3	3 of 8, 38% (1)	8 of 8, 100% (5)	7 of 8, 88% (4)
Weighted Score	**19**	**47**	**75**	**78**

C-005 function groups: authentication and sessions, email, data access and search, cart, favourites and order logic, image upload, admin CRUD pages, JSON validation and API, and dashboard aggregations. Stack Overflow counts were taken on **5th October 2026**.

C-004 mechanisms: enforced project layout, layering or dependency injection, ORM with migrations, and typed models.

5.8.7. Close-score check

Spring Boot scored 78, Django 75 and FastAPI 47. Django scored 3 points below Spring Boot, so the gap between the top two is small. Spring Boot led on C-001 (team experience): its Used multiplier is 1.00 (score 4), against 1.38 (score 1) for Django. Django led on C-003 (311,126 Stack Overflow questions against 150,213) and on C-005 (8 of 8 groups against 7 of 8). The two tied on C-002 and C-004. The preference means were FastAPI 4.38, Spring Boot 4.00 and Django 3.38, and no preference differed by 2 or more points between us. Suitability and preference ratings do not enter the weighted score. They are used in this check and in the Justification.

5.8.8. Decision

Selected framework: Django 5.2 with Django REST Framework

5.8.9. Justification

Django scored 3 points below Spring Boot on the weighted comparison, so the scores alone do not decide it. We chose Django because it is a framework that includes batteries, meaning we will not have to work from scratch when rebuilding this project, furthermore SpringBoot is way too overcomplicated for a project with the time constraint we have. Django led on community support (C-003) and functional fit (C-005), and it tied with Spring Boot on C-002 and C-004. Spring Boot led on team experience (C-001). Django also had the lowest preference mean of the three (3.38).

5.8.10. Verify

ID	What we checked	Result	Evidence
G-001	Is the rebuilt backend easier to modify than the Express code?	Improved on the figures measured	Backend highest v(G) 19 (render_email in common/email_templates.py) against 66 in SE262. 13 of 353 functions above 10 (3.7%) against 45

			of 713. Code smells 0.43 per KLOC against 9.18 for the whole rebuild (scopes differ, Section 8.1)
G-002	Is the backend tested and low in defects?	Improved on the figures measured	578 backend tests. Line coverage 98.7% and branch coverage 91.8%, 0 bugs, quality gate passed on dev (SonarQube Cloud, 15 October 2026)
G-003	Does the backend do everything the old one did?	To be confirmed against the Section 3.1 checklist	docs/api-contract.md lists every endpoint with role, request, response and error codes
K-001	Does the framework differ from Express?	Met	Django 5.2 with Django REST Framework replaces Node.js and Express
K-002	Does the backend support the required functions of the store?	To be confirmed against the Section 3.1 checklist	Same checklist as G-003
K-003	Can SonarQube analyse the code and its coverage?	Met	coverage.py output was imported into SonarQube Cloud, 98.7% line coverage (dev branch, 15 October 2026)
K-004	Was it finished in time?	Yes we finished on time	The dev branch has 286 commits from 24 September to 14 October 2026, built by two people

5.8.11. Validate

We built the backend with Django 5.2 and Django REST Framework. It has six apps (accounts, catalog, campaigns, cart, orders, analytics) plus a shared common package, with 21 models, 8 migration files and 578 backend tests. The 353 backend functions have a highest v(G) of 19 and 13 are above 10 (3.7%), against 66 and 45 of 713 in SE262. The quality gate passed on dev. One prediction did not hold: Django authentication did not carry over, because we wrote custom sessions. See Section 8.3.

5.9. Database Framework (Original MySQL)

5.9.1. Options

Technology

- Option A: PostgreSQL
- Option B: MongoDB
- Option C: SQLite

Overview

- Option A: Open-source relational database with SQL, ACID transactions, rich data types (including JSON) and strong standards compliance; runs as a separate server.
- Option B: Document-oriented NoSQL database that stores flexible JSON-like documents without a fixed schema; runs as a separate server.
- Option C: Lightweight relational database stored in a single file and embedded in the application, with no separate server to run.

5.9.2. Strengths and weaknesses

Option A: PostgreSQL

- **Strengths:** Full relational model with foreign keys, joins and transactions, which fits the store's linked data (orders, order items, products, customers). Mature, well-documented, and supported by every framework under consideration.
- **Weaknesses:** Needs a separate server to install, configure and keep running, which adds setup work for a two-person team. Strict schemas mean structural changes need migrations.

Option B: MongoDB

- **Strengths:** Flexible schema, so product data with variable attributes (variants, translations) can sit in one document, and structural changes need no migrations. Handles JSON data naturally and is easy to start with.
- **Weaknesses:** Relationships such as orders to products are not enforced and joins are limited, so consistency must be handled in application code. Moving from the relational MySQL schema requires remodelling the data.

Option C: SQLite

- **Strengths:** Zero configuration, with no server and a single file, which makes setup and automated tests very quick. Full SQL, so the existing relational structure and queries carry over with little change.
- **Weaknesses:** Only one writer at a time and fewer features than server databases, so it is less suited to a deployed multi-user store. A file database needs care for hosting and backups.

5.9.3. Suitability for project goals

Strong, Partial, Weak, or None will be used to indicate suitability for the project goals. Goal IDs (G-XXX) refer to the tables in Section 2.1.

Goal	PostgreSQL	MongoDB	SQLite
G-001	**Strong**. An explicit schema with constraints and migration tools makes changes traceable and keeps data consistent as the code changes.	**Partial**. A flexible schema makes early changes easy, but structure is enforced only in application code, so inconsistencies can build up.	**Strong**. Same relational schema and migration tooling as other SQL databases, and the single-file setup keeps the environment simple.
G-002	**Strong**. ACID transactions and constraints prevent invalid data, and tests can run against a real test database.	**Partial**. Tests need a running MongoDB or a substitute, and without foreign keys invalid references must be caught by application-level tests.	**Strong**. In-memory or temporary file databases make automated tests fast and need no extra services.
G-003	**Strong**. Joins, transactions and	**Partial**. All the data can be stored, but	**Strong**. Supports the relational queries the

	aggregates cover orders, the cart and the admin dashboard summary.	order-to-product relationships and dashboard summaries need aggregation pipelines or extra application logic.	store needs, although limited concurrent writes could matter in a deployed store.

5.9.4. Suitability for personal preferences

Each of us rates P-001 to P-004 separately from 1 (low) to 5 (high). The Team row is the average of the two ratings. A difference of 2 or more points between us on a preference is noted below the table.

Option	**Rater**	**P-001**	**P-002**	**P-003**	**P-004**	**Mean**
PostgreSQL	VK	4	5	4	4	4.25
	Team Member	5	5	4	4	4.5
	Team	4.5	5	4	4	4.38
MongoDB	VK	3	3	2	4	3
	Team Member	2	4	2	4	3
	Team	2.5	3.5	2	4	3
SQLite	VK	3	5	4	4	4
	Team Member	3	4	3	3	3.25
	Team	3	4.5	3.5	3.5	3.63

Differences of 2 or more points: none.

5.9.5. Team experience

Team experience is rated with the COCOMO levels in Section 4.4, using the lower rating where we differ.

Framework	**Rater**	**LEXP**	**VEXP**	**Effort multiplier**	**Basis**
PostgreSQL	VK	High (0.95)	High (0.90)	0.86	Used in many projects, and deployed and maintained in more than one.
	Team Member	Normal (1.00)	Normal (1.00)	1.00	Used and deployed it in a course project.
	Used	Normal (1.00)	Normal (1.00)	1.00	Lower of the VK and Team Member ratings for each driver.
MongoDB	VK	Low (1.07)	Low (1.10)	1.18	Followed a tutorial and played around locally.
	Team Member	V.low (1.14)	V.low(1.21)	1.38	Never used the framework before.
	Used	V.low (1.14)	V.low (1.21)	1.38	Lower of the VK and Team

					Member ratings for each driver.
SQLite	VK	Low (1.07)	Low (1.10)	1.18	Followed a tutorial and played around locally.
	Team Member	Low (1.07)	Low (1.10)	1.18	Followed a tutorial and played around locally.
	Used	Low (1.07)	Low (1.10)	1.18	Lower of the VK and Team Member ratings for each driver.

Effort multiplier is the LEXP multiplier times the VEXP multiplier. The Used row is the one that feeds C-001.

5.9.6. Weighted comparison

Each cell shows the raw measure and, in brackets, the score from Section 4.3. The weighted score is the sum of weight times score, with a maximum of 95.

Criterion	Weight	PostgreSQL	MongoDB	SQLite
C-001	4	1.00 (4)	1.38 (1)	1.18 (2)
C-002	5	4 of 4 (5)	2 of 4 (3)	4 of 4 (5). Based on documentation
C-003	3	178,778 questions	175,156 questions	94,920 questions

		(5)	(3)	(1)
C-004	4	4 of 4 (5)	1 of 4 (2). Based on documentation	3 of 4 (4). Based on documentation
C-005	3	4 of 4, 100% (5)	2 of 4, 50% (2)	3.5 of 4, 88% (4). Based on documentation
Weighted Score	**19**	**91**	**42**	**64**

C-002 requirements: Product (database), external. Test support, requirements met out of 4 (the test runner can create and drop a real test database automatically, it works with the backend test framework without mocks, each test is rolled back, it runs in CI), count.

C-004 mechanisms: Product (database), external. Structure support, mechanisms met out of 4 (enforced schema with constraints, foreign keys, migration tooling supported by the chosen backend, typed columns), count.

C-005 data needs: Product (database), external. Functional fit, share of the store data needs supported, percent. The needs are transactional order placement with stock locking, relational joins across catalogue, cart and orders, dashboard aggregations, and text search and filtering.

Stack Overflow counts were taken on **14 October 2026**. The PostgreSQL and MongoDB counts are close. MongoDB and SQLite were not exercised, so their other scores are based on documentation.

5.9.7. Close-score check

PostgreSQL scored 91, SQLite 64 and MongoDB 42, so the margin between the top two is 27 points and the result is not close. PostgreSQL leads on C-001 (score 4, against 1 for MongoDB and 2 for SQLite) and on C-003 (score 5, against 3 for MongoDB and 1 for SQLite), and the PostgreSQL and MongoDB Stack Overflow counts (178,778 and 175,156) are close. It also leads on C-004 and C-005. The

preference means were PostgreSQL 4.38, SQLite 3.63 and MongoDB 3.00, and no preference differed by 2 or more points between us. Suitability and preference ratings do not enter the weighted score. They are used in this check and in the Justification. MongoDB and SQLite were judged on documentation only, and PostgreSQL was used and tested.

5.9.8. Decision

Selected database: PostgreSQL 16

5.9.9. Justification

We chose PostgreSQL 16 because it scored highest (91, against SQLite 64 and MongoDB 42). It led on C-001 (Used multiplier 1.00, score 4) and C-003 (178,778 Stack Overflow questions), and it had the highest preference mean (4.38). Its relational model with foreign keys, joins and transactions fits the linked store data. It is the only option we built and tested, so its C-002, C-004 and C-005 figures come from our own work, while the MongoDB and SQLite figures are based on documentation. SQLite is the runner-up with 64 points, but its documented weakness is that select_for_update() has no effect there, which matters for stock locking at checkout.

5.9.10. Verify

ID	What we checked	Result	Evidence
G-001	Do the schema constraints keep data consistent and changes traceable?	Met for the schema	21 tables with foreign keys and unique constraints, 8 Django migration files, ERD in se327/docs/erd.md
G-002	Can tests run against a real database?	Met	All 578 backend tests run on a real PostgreSQL 16 with no database mocks, locally and in CI
G-003	Does the database support the store data needs?	To be confirmed against the	Order placement locks stock rows

		Section 3.1 checklist	with select_for_update (13 places) inside transactions (transaction.atomic, 23 places). Dashboard figures use annotate with Sum and Count
K-001	Does the database differ from the original MySQL?	Met	PostgreSQL 16 replaces MySQL
K-002	Does the database support the required functions of the store?	To be confirmed against the Section 3.1 checklist	Same checklist as G-003
K-003	Can the tests feed coverage into SonarQube?	Met	coverage.py output was imported into SonarQube Cloud, 98.7% line coverage (dev branch, 15 October 2026)
K-004	Was it finished in time?	Yes we finished on time	The dev branch has 286 commits from 24 September to 14 October 2026, built by two people

5.9.11. Validate

We built the database as PostgreSQL 16 with a relational schema of 21 tables, foreign keys and unique constraints, described in an ERD in se327/docs/erd.md. Orders keep price and name snapshots, so order history survives product deletion. Stock changes use row locks inside transactions. All backend tests run against a real PostgreSQL, with no mocks of the database, both locally and in CI. We have not yet checked every function in Section 3.1 against the rebuild.

6.

7. Selected Tech Stack

The original and chosen technology for each component. The reasons for each choice are in Section 5.

Component	**Original (SE262)**	**Chosen**	**Description**
UI framework	Bootstrap 5	Tailwind CSS	Utility-first CSS framework for styling.
Frontend framework	EJS	React 19 with Vite	Single-page app that the browser loads and that calls the JSON API.
Backend framework	Node.js and Express	Django 5.2 with Django REST Framework	JSON API under /api with views, ORM, migrations and custom session authentication.
Database	MySQL	PostgreSQL 16	Relational database with 21 tables, foreign keys and unique constraints.

7.1. How do the four frameworks fit together?

The browser loads the React app, which is styled with Tailwind CSS. The app calls a JSON API under /api that is served by Django views. The Django ORM maps the data to PostgreSQL. Sessions are rows in the database, and the browser holds an HTTP-only cookie. Uploaded product images go to Supabase Storage. In Docker Compose, an nginx container serves the app and proxies /api to Django, so the whole system starts with one command.

7.2. Effort estimates

Component	Owner	Estimated hours	Actual hours
UI framework	Team Member	10	8.7
Frontend framework	Team Member	25	18.3
Backend framework	VK	10	11
Database	VK	5	3

Actual hours are the time we each spent working, from our own estimate of the days worked.

8. Process Evidence

8.1. Git Workflow

We use issue templates with labels by type (feature, bug, chore) and by area (backend, frontend and others). Each feature has one issue, one branch and one pull request into dev, and the other teammate reviews it. Commits are small and modular. As of 14 October 2026 the dev branch has 286 commits, from 24 September to 14 October, and the repository has 54 pull requests (54 merged), and 51 issues (51 closed).

The frontend bug-fix pull requests got the same review. It found problems such as a price edit that was silently dropped when the SKU field was changed, a language switch that wiped the typed checkout form, and a return-to-page link that failed on a control character. These were fixed with tests before merge.

Every backend pull request also got an independent review pass looking for defects. It looked for race conditions on stock, account enumeration through timing, lost updates, unsafe uploads and wrong cleanup on delete. Findings were fixed with tests before merge. Two examples: the hard delete of product images missed files in a hosted bucket, and one test did not exercise the code it claimed to. The frontend bug-fix pull requests got the same review, which caught for example a price edit that was silently dropped when the SKU field was changed and a language switch that wiped the typed

checkout form. The repository is github.com/guykongthongcmu/software-metrics-project.

8.2. SonarQube

We use two SonarQube Cloud projects, one for the SE262 baseline (private) and one for the rebuild, so the before and after figures come from the same tool. Analysis runs from GitHub Actions on every push and pull request. It imports coverage.py XML for the backend and lcov for the frontend, and a quality gate is applied to pull requests. Other static checks are oxlint, tsc with checkJs and vitest. We also hardened CI: GitHub Actions are pinned by commit SHA, pip installs binary-only packages with a pinned coverage version, and npm runs with install scripts disabled.

8.3. Statement & Branch Test Coverage

Metric	**SE262 baseline**	**Rebuild**
Statement (line) coverage	0.0%	98.7%
Branch coverage	0%	91.8%
Overall coverage	0%	96.7%
Automated tests	none	578 backend, 345 frontend

Coverage is imported into SonarQube Cloud from coverage.py XML for the backend and lcov for the frontend. Frontend only (Vitest, v8 provider): 96.93% statements, 91.89% branches, 94.29% functions, 98.99% lines, from 345 tests. Figures are from the dev branch analysis of 15 October 2026.

8.4. Quality Gate

Item	**Status on dev**
Quality gate	Passed
Security rating	A
Reliability rating	A

Maintainability rating	A
Vulnerabilities	0
Security hotspots	0
Duplicated lines	0.0%

The quality gate is applied to pull requests. Some pull requests were blocked by it and fixed before merge. For example, pull request 57 had a SonarQube security finding. Earlier in the project the backend had 14 SonarQube findings (cognitive complexity in 5 functions, a slow email regular expression and small cleanups). All were fixed in pull requests 84, 85 and 86. The frontend had 28 findings on 14 October (25 admin colour contrast, 2 handlers on non-interactive elements, 1 nested conditional), fixed in pull requests 102 and 104. 11 !important rules in the admin CSS were removed later. Figures are from the analysis of 15 October 2026.

9. Reflection

9.1. Before and After Measurements

The table compares the SE262 baseline with the rebuild. It uses the same metrics as Section 3.2.

Metric	**SE262 baseline**	**Rebuild**	**Goal checked**
Statement coverage	0.0% (no tests exist)	98.7%	G-002
Branch coverage	0% (no tests exist)	91.8%	G-002
Bugs per KLOC	0.62 (8 bugs)	0.00 (0 bugs)	G-002
Highest v(G)	66 (updateAdminProduct in product.service.js)	Backend 19 (render_email in common/email_templates.py). Frontend 33 (ProductDetailPage.jsx)	G-001
Methods with v(G) above 10	45 of 713 methods	Backend 13 of 353 functions (3.7%). Frontend 35 of 881 functions (4.0%)	G-001
Code smells per KLOC	9.18 (119 code smells)	0.43 (5 code smells)	G-001
Size (KLOC)	12.97 (12,966 non-comment lines)	11.56 (11,555 non-comment lines)	context for the per-KLOC rates

The scopes differ. The baseline is 12.97 KLOC of JavaScript only. The rebuild is 11.56 KLOC of Python, JavaScript and CSS. The rebuild figures are from SonarQube Cloud, dev branch, analysis of 15 October 2026. The v(G) figures come from the lizard tool for both, with tests, migrations and dependencies excluded for the rebuild. Lizard does not parse React JSX arrow functions (it found only 19 functions in the frontend), so the frontend figures were measured with ESLint's complexity rule instead (Section 5.2). For context only, SonarQube

reports a total cyclomatic complexity of 3,011 over 1,205 functions for the whole rebuild.

9.2. Structure: Coupling and Cohesion

Coupling is counted as import lines between the six Django apps in non-test code. These figures are import counts, not a formal coupling metric.

App	Imports from other apps (import lines)
accounts	none
analytics	accounts 1, cart 1, catalog 1, orders 1
campaigns	accounts 2, catalog 1, orders 1
cart	accounts 2, catalog 6
catalog	accounts 4, analytics 1
orders	accounts 3, cart 1, catalog 1

Frontend (48 source files). The three files in lib (API client, language and login redirect) import nothing from other groups. The admin pages (10 files) import only from lib (9 import lines), the account pages (11 files) only from lib (8), and the storefront pages (22 files) from lib (19) plus one import from the account pages. Only App.jsx and main.jsx import across the page groups, to register routes. 27 of the 48 files use the shared API client. Again these are import counts, not a formal coupling metric. For cohesion, each page group holds one area of the site and shares only the lib helpers.

The only two-way pair is catalog and analytics. Shared utilities (error envelope, email, storage) live in the common package. For cohesion, each app groups one business area and has its own models, services, views and urls.

Frontend (48 source files): lib (3 files) imports no other group. The admin, account and storefront pages import only from lib (9, 8 and 19 import lines), plus one storefront import from the account pages. Only App.jsx and main.jsx import across page groups, to

register routes. These are import counts, not a formal coupling metric.

9.3. Strengths and Weaknesses: Expected VS Experienced

The table compares what Sections 5.3 and 5.4 predicted with what happened in the build.

Predicted in Section 5	**What happened**
Tailwind: highly customizable and flexible, so designs aren't limited to predefined components.	Custom CSS defines the Core & Co theme and storefront styles, with dedicated styles for checkout and the legacy admin dashboard, orders, products, categories, and campaign builder. These styles preserve their page layouts, responsive behavior, and UI states alongside Tailwind classes.
Tailwind: no ready-made components, so they must be built manually or added from other libraries.	Built in React and styled with Tailwind utility classes and project-specific CSS. No external UI component library was added.
React: focuses on the UI layer, so routing, state management and other features need extra libraries.	React Router DOM 7.18.4 handles routing. State is managed with React's built-in state and context: page-level state uses hooks, and FavoritesContext shares favorite data across components. This adds a router setup and a small context provider; no separate state-management library is used.
React: the large number of libraries makes technology choices more complex.	Not a problem in practice. We kept the dependency list short (React Router DOM and Flatpickr) and picked well-known libraries. Because the SE262 legacy code already showed what each page needed, we knew what we wanted before choosing, so choosing did not slow the work.
Django: batteries included, with authentication and an admin panel.	Authentication did not carry over. We wrote custom session authentication (an opaque token in a session table and an HTTP-only

	cookie, with separate customer and admin guards) to keep customers and admins separate. We did not use django.contrib.auth sessions. The admin panel was not used.
Django: ORM and migrations.	As predicted. The backend has 21 models and 8 migration files. We used transaction.atomic in 23 places and select_for_update in 13 places for cart and checkout locking. There is no raw SQL.
Django: WSGI copes less well with many concurrent requests.	Not a problem in our testing.
Django: the REST layer requires adding Django REST Framework.	We added it and use it for views, parsers and permissions only, with no serializers or viewsets.
PostgreSQL: full relational model with foreign keys, joins and transactions fits the linked data.	As predicted. The schema has 21 tables with foreign keys and unique constraints. Stock changes use row locks inside transactions. Orders keep price and name snapshots, so order history survives product deletion.
PostgreSQL: needs a separate server, and strict schemas need migrations.	We run everything with Docker Compose, started with one command. Schema changes went through Django migrations. The separate server was not difficult to add since there was a lot of supporting documentation.

9.4. Re-measuring the selection criteria

We only re-checked what we can measure now. C-002: coverage.py output for the backend and lcov output for the frontend were imported into SonarQube Cloud, and the quality gate passed on dev. C-004: the backend uses the Django ORM with migrations (8 migration files), and each app has its own models, services, views and urls. The frontend is split into admin, account and storefront pages with one shared API client, and types are checked by tsc. C-001: the Used multipliers were set before the build and should be re-rated now. New ratings: Django VK: 1.38, Team Member: 1.38, PostgreSQL

VK: 1.00, Team Member: 1.00, Tailwind CSS VK: 0.95, Team Member: 0.95, React VK: 1.00, Team Member: 1.00.

9.5. Goals: Were they met?

G-001 (maintainability). The backend improved on the figures we could measure. The highest v(G) fell from 66 to 19, functions above 10 fell from 45 of 713 to 13 of 353 (3.7%), and code smells fell from 9.18 to 0.43 per KLOC. On the frontend, measured with ESLint's complexity rule, the highest v(G) is 33 and 35 of 881 functions (4.0%) are above 10. The backend and frontend tools differ, so the figures are not directly comparable, and the scopes differ from the baseline, see Section 8.1.

G-002 (reliability). Line coverage went from 0% to 98.7% and branch coverage to 91.8%. Bugs fell from 8 to 0. The rebuild has 578 backend tests and 345 frontend tests, and the baseline had none. The quality gate passed on dev.

G-003 (functionality). Pending. We still have to check each function in Section 3.1 against the rebuild, so we do not claim that every original function works. On the frontend, 23 of 23 legacy storefront templates have a React route and 38 of 38 distinct transactions are traced to a route and a test (with a mocked API). The nine data groups (FP-001 to FP-009) are checked in the backend and database sections.

9.6. Recommendation for a similar future project

We would run SonarQube on the old code and the new code with the same tool, as we did with two SonarQube Cloud projects. The before and after figures then come from one source. We would keep the analysis in CI and apply the quality gate to pull requests. Pull request 57 was blocked by a SonarQube security finding and was fixed before merge. We would also give each pull request an independent review for defects. In this project the reviews found problems such as a hard delete of product images that missed files in a hosted bucket, and a test that did not exercise the code it claimed to. We would check that the complexity tool can parse the frontend language first, because lizard could not measure our React code. We used ESLint's complexity rule for the frontend.

9.7. Impact on process, product and learning

Process - We worked with one issue per feature and one branch and pull request per issue into dev. The other teammate was the reviewer, and an AI assistant occasionally filled in when neither of us was available. The SonarQube quality gate on every pull request changed how we worked. It blocked pull requests until we fixed findings such as a security issue and a slow regular expression, and every backend pull request also got an independent review that found real defects before merge.

Product - Line coverage rose from 0% to above 98%, bugs fell from 8 to 0, and the backend has 578 tests that all pass. The store runs with one Docker Compose command, and uploaded images go to Supabase Storage. It also has limits. The language selector changes only the data from the API, not the fixed page text. The admin chat feature from SE262 was not rebuilt as it was not properly implemented in SE262 as well.

Learning - Django ORM was interesting to learn about, especially on how and where to use it. Tailwind was also very easy to use because it replaces custom CSS files with pre-built, single-purpose utility classes applied directly inside the HTML. Together we learned that a feature that looks built in may not fit the design, and that figures from a tool are only useful when they are measured the same way on both sides.